

Smart MCQ Solver Challenge

Deep Learning and Generative AI — Project Report

Sagar K Chaudhary | Roll No: 23f2002523 | BS in Data Science and Applications, IIT Madras

1. Abstract

This project addresses the Smart MCQ Solver Challenge, in which each item consists of a question and five candidate answers with exactly one correct, and the required output is the three most likely answers in ranked order, scored by Mean Average Precision at 3 (MAP@3).

Four models were developed: a custom Siamese BiGRU dual-encoder built from scratch, a pretrained MiniLM sentence-transformer used zero-shot, a DeBERTa-v3 transformer fine-tuned with LoRA, and an ELECTRA model from a different pretraining family. The fine-tuned DeBERTa reached 0.9983 validation MAP@3 while training only 0.79% of its parameters.

The defining result was diagnostic rather than architectural. The from-scratch model reached a perfect validation score, prompting an experiment in which a model structurally incapable of reading the question was trained on the answer options alone. It also reached 100% accuracy, establishing an annotation artifact in the dataset. A diagnostic leaderboard submission scoring 0.75311 confirmed the artifact transfers to the hidden test set; the finding was reported to and confirmed by the course TA.

Six single-model configurations, spanning 644 thousand to 400 million parameters, all scored between 0.749 and 0.753 — a spread of roughly two test rows. The plateau was traced to every model learning the same artifact-driven rule, evidenced by mutual prediction agreement above 99%. Blending across pretraining families, using an ELECTRA model agreeing with DeBERTa on only 83% of rows, produced the only improvement exceeding the resolution of the metric: a final public leaderboard score of **0.75644**.

2. Introduction

2.1 Problem statement

Each row contains a question (the prompt) and five candidate answers labelled A to E, exactly one of which is correct. The task is to rank the options and submit the top three. With a single correct answer per question, MAP@3 reduces to a reciprocal-rank score: 1.000 if the correct answer is ranked first, 0.500 if second, 0.333 if third, and zero if absent. The final score is the mean across rows. Uniform random guessing yields 0.3578; the qualifying cutoff is 0.73.

2.2 Project objectives

1. Exceed the 0.73 qualifying threshold on the Kaggle leaderboard.
2. Build and compare at least three distinct model types — a fine-tuned transformer, a custom deep learning architecture, and a recurrent model — with all runs tracked on Weights and Biases.
3. Gain practical command of the Hugging Face ecosystem, PyTorch, and parameter-efficient fine-tuning with LoRA.
4. Understand the dataset well enough to explain, rather than merely report, the behaviour of every model built.

2.3 Report structure

Section 3 covers the dataset, exploratory analysis and preprocessing. Section 4 describes the tokenization strategy. Section 5 presents the three required model types with architectures and training strategy, together with the artifact investigation that shaped every later decision. Section 6 gives comparative performance and Kaggle results. Section 7 concludes with learnings, challenges and future work, and Section 8 lists references.

3. Dataset and Preprocessing

3.1 Dataset description

Property	Training set	Test set
Rows	2,000	500
Columns	8 (id, prompt, A–E, answer)	7 (answer withheld)
Missing values	0	0
Mean prompt / option length	117 / 165 characters	—
Unique vocabulary	approx. 2,975 words	—

Table 3.1 — Dataset dimensions. Every column is textual; there are no numerical features.

Because all fields are text, there is no imputation, no categorical encoding and no hand-crafted feature engineering. Representations are learned by the models themselves; the only required transformation is converting text into token indices.

3.2 Exploratory data analysis

Correct answers are distributed B=490, C=459, A=369, D=358, E=324. The majority class covers only 24.5% of rows, so there is no exploitable label shortcut. The mild imbalance is why macro F1 is reported alongside accuracy, since it penalises neglect of the rarer option E.

Text lengths were measured to fix a concrete hyperparameter. A typical (prompt, option) pair is roughly 70 tokens, but the distributions have long tails: the longest prompt is 337 characters and the longest option 662, giving a worst case near 250 tokens. A maximum sequence length of 256 was chosen to sit just above this. The five option-length distributions overlap almost completely, so length alone does not identify the correct answer — the correct option is longest in about 41% of rows against 20% by chance.

Observation. All five options appear as the correct answer with comparable frequency, so accuracy above 20% must come from reading the text rather than exploiting the label prior. The option-length distributions overlap, ruling out length as a trivial giveaway.

Before committing to any architecture, the intuitive hypothesis that the correct option is the one most similar to the question was tested with TF-IDF and cosine similarity. It scored 0.1635 top-1 accuracy, below the 0.20 random baseline, meaning similarity is anti-correlated with correctness: distractors are deliberately written on the question's topic and reuse its vocabulary, while the correct answer often introduces new terminology. This inexpensive experiment redirected all subsequent effort towards architectures that read question and option jointly.

3.3 Data preprocessing

Templated boilerplate such as “Pick the best possible answer:” and “among the listed options.” was found across all 2,500 prompts from both files, with three distinct trailing templates each occurring in roughly 400 or more rows. Text identical across hundreds of rows carries no discriminative information and only consumes the token budget. These wrappers were removed with regular expressions and whitespace normalised, reducing mean prompt length from

117 to 91 characters. The identical cleaning function was applied to both files, eliminating train-test drift, and the raw column was preserved rather than overwritten.

3.4 Data augmentation

One technique was applied experimentally: option-order shuffling, permuting the five options within each training batch and moving the label to follow the correct option. Its purpose was mitigation of the artifact described in Section 5.4 rather than regularisation. It did not improve the leaderboard score and was not used in the final model, but produced a diagnostically valuable negative result (Table 5.3).

4. Tokenization Strategy

4.1 Primary tokenizer and justification

The best-performing model uses the DeBERTa-v3 tokenizer, a SentencePiece Unigram model with approximately 128,000 subword tokens, distributed with microsoft/deberta-v3-base (He et al., 2021) — a 12-layer encoder pretrained with replaced-token detection and equipped with disentangled attention. Three properties justify the choice:

1. Subword coverage. A word-level vocabulary built from this dataset contains only 2,975 words, so any unseen test word collapses to a single unknown token — an unseen technical term and a meaningless string become identical inputs. A subword vocabulary decomposes rare words into known fragments, so no input is ever fully unknown.
2. Native sentence-pair support. The tokenizer accepts two sequences and emits [CLS] question [SEP] option [SEP] with the corresponding token_type_ids, exactly the format DeBERTa saw during pretraining.
3. Alignment with the checkpoint. Using the paired tokenizer guarantees token indices match the pretrained embedding table; a mismatched tokenizer would silently produce meaningless embeddings.

4.2 Implementation details

Tokenization is applied through the datasets library using .map(), which processes rows in batches with caching. Each row is expanded into five (question, option) pairs. Truncation is enabled at max_length = 256, selected from Section 3.2. Padding is dynamic: a custom collator flattens the five choices per row, pads each batch only to its own longest sequence, and reshapes to (batch, 5, sequence_length) — the shape a multiple-choice head requires, and cheaper than always padding to 256.

4.3 Experimental comparison

Tokenizer	Type	Vocabulary	Outcome
Custom word-level	regex word split	2,975	unknown-word collapse; from-scratch model only
TF-IDF n-gram	count-based vectors	corpus-derived	0.1635 accuracy, below random
MiniLM WordPiece	subword	approx. 30,000	0.2567 accuracy zero-shot
DeBERTa-v3 SentencePiece	subword unigram	approx. 128,000	0.9983 validation MAP@3 (selected)
ELECTRA WordPiece	subword	approx. 30,000	0.8900; retained for ensemble diversity

Table 4.1 — Tokenization pipelines evaluated. Count-based vectors performed below random, confirming surface word overlap misleads on this dataset.

5. Modeling and Experimentation

Section	Model	Required category
5.1	DeBERTa-v3-base + LoRA	Fine-tuned transformer
5.2	Siamese BiGRU dual-encoder	Custom deep learning model
5.3	Bidirectional GRU encoder (within 5.2)	RNN-based model
5.5	ELECTRA-base + LoRA	additional, for ensembling

Table 5.1 — Mapping of models to the required categories. A fifth prompt-blind model was built for diagnosis only.

5.1 Model 1 — Fine-tuned transformer: DeBERTa-v3 + LoRA

The model is microsoft/deberta-v3-base (He et al., 2021; Hugging Face model card: huggingface.co/microsoft/deberta-v3-base), a 12-layer encoder with hidden size 768 and 12 attention heads, wrapped in AutoModelForMultipleChoice, which applies a shared classification head to each of the five encoded pairs and emits one logit per option.

Salient architectural points. Disentangled attention represents each token's content and relative position separately, which helps distinguish which claim attaches to which entity when distractors share vocabulary with the question. Replaced-token-detection pretraining yields a discriminative encoder well matched to a scoring task. Most decisively, sentence-pair encoding places question and option in one sequence separated by [SEP], so self-attention compares their tokens directly — the architectural difference from a bi-encoder, which embeds the two texts independently and cannot perform this comparison.

Fine-tuning strategy. LoRA (Hu et al., 2021) freezes the pretrained weights entirely and injects trainable low-rank matrices beside the attention projections. An update to a 768 x 768 weight matrix (589,824 values) is factorised into 768 x 16 and 16 x 768 matrices totalling 24,576 values, a 96% reduction. Only these adapters and the randomly initialised classification head are trained.

Setting	Value	Justification
Trainable parameters	1,476,097 / 185,899,010 (0.79%)	frozen backbone retains pretrained knowledge
LoRA rank / alpha	r = 16, alpha = 32	adapter capacity; alpha = 2r by convention
Target modules	query_proj, key_proj, value_proj	attention projections steer what the model attends to
Layers frozen	all 12 encoder layers	adapters supply task-specific capacity instead
Optimizer / weight decay	AdamW, 0.01	standard for transformer fine-tuning
Learning rate	2e-4	an order above the 2e-5 of full fine-tuning: LoRA trains fresh matrices
Schedule / clipping	linear, warmup_ratio 0.1; max_grad_norm 1.0	added after an exploding loss (Section 5.6)
Epochs / batch size	3 / 4	validation metrics plateaued by epoch 3
Precision	float32	fp16 incompatible with LoRA modules_to_save (Section 5.6)

Table 5.2 — Fine-tuning configuration. No quantization was applied: LoRA alone brought training within available GPU memory.

5.2 Model 2 — Custom deep learning model: Siamese BiGRU dual-encoder

This model is built from scratch with no pretrained components, including the tokenizer. Its vocabulary is constructed by counting word frequencies in the training text, reserving index 0 for padding and index 1 for unknown words, and encoding every text to a fixed length of 80 indices.

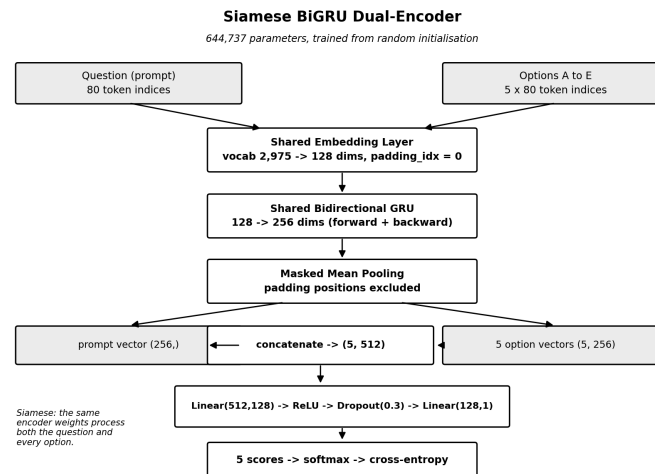


Figure 5.1 — Architecture of the custom Siamese BiGRU dual-encoder.

Design justification. The Siamese shared-weight design uses one encoder for both question and options, halving parameters and placing both texts in a single representation space so their vectors are directly comparable — a prerequisite for the concatenate-and-score head. Masked mean pooling averages over real tokens only; without the mask, padding would pull every short text towards zero, and mean pooling is preferred over the final hidden state because with padding that position is frequently meaningless. The concatenate-and-score head lets the model learn an interaction between question and option rather than a fixed similarity function — precisely the capability TF-IDF and MiniLM lack. At 644,737 parameters against 1,700 rows, a perfect held-out score should be implausible, which is what made the observed result diagnostic.

5.3 Model 3 — RNN-based model: bidirectional GRU encoder

The recurrent component of the architecture above constitutes the RNN-based model: a single-layer bidirectional Gated Recurrent Unit with input and hidden dimension 128, producing a 256-dimensional output per position after concatenating both directions.

Why a sequence model is appropriate. Correctness here depends on word order and on which entity a claim attaches to. Options within an item share almost all of their vocabulary and differ in how it is arranged — one asserts that humans exist within time, another that humans are time. A bag-of-words representation cannot separate these, which is exactly why TF-IDF scored below random. A recurrent encoder processes tokens in order and maintains a hidden state carrying context forward, so it can represent this distinction in principle.

Design choices. A GRU was chosen over a plain RNN because its update and reset gates mitigate the vanishing-gradient problem that prevents a simple recurrent network from learning dependencies across a full sentence, and over an LSTM because two gates against three gives roughly 25% fewer recurrent parameters — an advantage with only 1,700 examples. Bidirectionality means each position carries context from both sides, which matters when a claim's interpretation depends on a qualifier appearing later.

5.4 Diagnostic investigation: an annotation artifact

The custom model reached validation MAP@3 of 1.000 with 100% accuracy. A 644,737-parameter model trained on 1,700 rows cannot legitimately solve a reasoning task perfectly on held-out data, so this was treated as an anomaly requiring explanation. The hypothesis was that correct answers carry a writing-style signature identifying them independently of the question. This yields a falsifiable test: if the answer is identifiable from option text alone, a model that never receives the question should still succeed; if reasoning is genuinely required, it cannot beat 20%. A prompt-blind model was built with no prompt argument at all in its forward pass, using the identical encoder and split so that the only variable is the presence of the question.

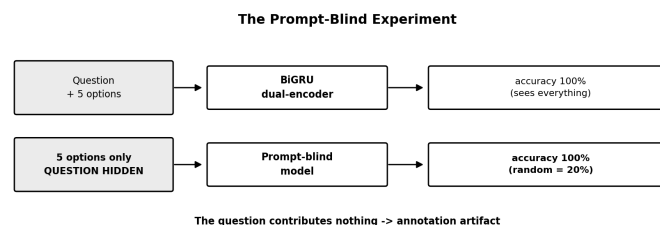


Figure 5.2 — The prompt-blind experiment. Removing the question entirely does not reduce accuracy.

The prompt-blind model reached 100% validation accuracy without ever reading a question, against a 20% random baseline. A diagnostic leaderboard submission built purely from this signal scored 0.75311, confirming the artifact is present in the hidden test set.

This is an annotation artifact: correct answers and distractors were evidently produced by different generation procedures, leaving a systematic stylistic difference. The finding was reported to the course TA, who confirmed it is a data problem and advised that the final model should not rely on the signal given a possible private leaderboard.

It is important to distinguish this from overfitting. In overfitting, training accuracy is high while validation accuracy falls; here validation accuracy is also perfect, so nothing was memorised. This is shortcut learning — the model learned a rule that genuinely generalises within this dataset, but the rule is “recognise the style of a correct answer” rather than “understand the question”.

Dependence was then quantified. Because a model adopts the most accessible signal available, a high validation score does not by itself demonstrate reasoning, so DeBERTa's predictions were compared against the prompt-blind detector. Agreement was 99.7% — across 300 rows the two models differ on approximately one. This explains the central discrepancy: the same model scores 0.9983 on validation and 0.75062 on the leaderboard, because validation is drawn from the artifact-rich training file while the test set is not equally exploitable.

Configuration	Agreement with prompt-blind	Public LB
Baseline (3-seed bagged DeBERTa)	99.7%	0.75187
Option-order shuffling	95.7%	0.73773
Label smoothing + cosine schedule	96.0%	0.74937

Table 5.3 — Both mitigation attempts reduced artifact reliance and both reduced the score. This is causal rather than correlational evidence: because the public test set contains the same artifact, moving away from it necessarily costs measured performance. It also establishes the artifact is textual rather than positional, since shuffling disrupts only position-dependent patterns.

5.5 Additional model — ELECTRA, and the ensembling breakthrough

After six configurations converged on the same score (Table 6.2), the plateau was diagnosed as every model learning the same rule. Weight searches over the existing models saturated at 1.0 on validation and gave no leaderboard benefit, which is expected: ensembling helps only when models fail on different rows, and mutual agreement here exceeded 99%. The diagnosis suggested its own remedy — introduce a model that disagrees. ELECTRA-base (Clark et al., 2020) is pretrained with replaced-token detection over a different objective mix, producing different inductive biases. It was trained through the identical pipeline, with the single change that its LoRA target modules use BERT-style names (query, key, value) rather than DeBERTa's *_proj names; supplying the wrong names attaches adapters to nothing and training silently learns almost nothing.

Model pair	Prediction agreement
DeBERTa vs prompt-blind detector	99.7%
DeBERTa seed-bagged vs 5-fold cross-validation	approx. 99% (same family)
DeBERTa vs ELECTRA	83.0%

Table 5.4 — ELECTRA is the first genuinely complementary model in the project.

Agreement was measured before any submission was spent, converting a guess into a testable prediction. The validation weight sweep was flat from 0 to 0.5 because DeBERTa's high confidence dominates the averaged probabilities, so 0.25 was chosen on the reasoning that a model at 0.89 MAP@3 merits meaningful but minority influence, then verified on the leaderboard.

5.6 Engineering challenges during training

1. Exploding loss. The first run reported a training loss of 543, where cross-entropy over five classes should begin near $\ln(5) = 1.61$. The cause was the freshly initialised classification head producing very large gradients in the opening steps. Adding warmup (ratio 0.1) and gradient clipping (1.0) resolved it; subsequent runs begin at approximately 1.58, which then served as a health check for every later run.
2. Mixed precision failures. bf16 was rejected by the runtime on the initial GPU, which lacks hardware bfloat16 support. fp16 raised “Attempting to unscale FP16 gradients”, because parameters in LoRA's modules_to_save are cast to half precision and the AMP gradient scaler refuses them. DeBERTa was therefore trained in float32, affordable precisely because LoRA trains only 0.79% of parameters. On the T4 used for ELECTRA, bf16 works and was enabled.

6. Performance and Comparative Analysis

6.1 Evaluation metrics

MAP@3 is the competition metric and primary criterion. Accuracy is reported as the rank-1 hit rate, and macro F1 as the class-balanced harmonic mean of precision and recall, which exposes neglect of rarer options that accuracy would partially conceal. All three are computed by a single shared function, unit-tested against a hand-computed case returning exactly 0.75, so every figure below is directly comparable.

6.2 Training performance

Training logs can be found on wandB.

6.3 Comparative report

Model	MAP@3	Accuracy	Macro F1	Train time
Random baseline	0.3578	0.1867	0.1859	—
TF-IDF similarity	—	0.1635	—	seconds
Custom BiGRU (from scratch)	1.0000	1.0000	1.0000	~1 min
MiniLM (pretrained, zero-shot)	0.4083	0.2567	0.2515	none
DeBERTa-v3 + LoRA	0.9983	0.9967	0.9966	~6 min
DeBERTa + option shuffling	0.9667	0.9567	0.9543	~6 min
DeBERTa 3-seed bagged	0.9872	0.9767	0.9756	~18 min
ELECTRA + LoRA (2-seed bagged)	0.8900	0.8267	0.8279	~8 min
Prompt-blind (diagnostic)	1.0000	1.0000	1.0000	~1 min

Table 6.1 — All models on the identical held-out validation split. Every figure must be read alongside the Section 5.4 caveat: validation carries the same artifact as training, so these numbers overstate genuine reasoning ability. The prompt-blind row is included deliberately as a reference point — any model close to it is suspected of relying on the same shortcut.

Trade-offs. The custom BiGRU trains in a minute against six for DeBERTa, but its apparent advantage is entirely artifact. DeBERTa-large has three times the parameters and took substantially longer, yet scored 0.74937 against 0.75062 — capacity was not the limiting factor. MiniLM required no training and delivered 0.4083, while six minutes of LoRA fine-tuning delivered 0.9983, quantifying what task-specific adaptation contributes over general-purpose embeddings.

6.4 Kaggle performance

Approach	Public LB	Outcome
Prompt-blind diagnostic (artifact only)	0.75311	excluded per TA guidance
DeBERTa-base + LoRA, single model	0.75062	baseline reasoning model
DeBERTa-base + LoRA, 3-seed bagged	0.75187	best single-family result
DeBERTa-large + LoRA	0.74937	3x capacity, no gain
5-fold cross-validation bagging	0.74930	all 2,000 rows used, no gain
Label smoothing + cosine schedule	0.74937	agreement fell to 96%, score fell
Option-order shuffling	0.73773	mitigation reduced score

Approach	Public LB	Outcome
DeBERTa + ELECTRA blend, w = 0.25	0.75644	FINAL SUBMISSION
DeBERTa + ELECTRA blend, w = 0.40	0.75602	equivalent within noise
Three-way blend including RoBERTa	0.74896	a third family did not help

Table 6.2 — Complete record of leaderboard submissions.

Final public leaderboard score: 0.75644 (qualifying cutoff 0.73; random baseline 0.3578).

Are these differences real? With 500 test rows, one row is worth 0.002 MAP@3, which sets the resolution of the metric. Six configurations spanning 644 thousand to 400 million parameters all landed between 0.749 and 0.753 — a spread of 0.004, roughly two rows. A six-hundred-fold difference in capacity producing no measurable difference is decisive evidence that capacity was never the bottleneck. The only change to exceed this band was cross-family blending (0.75187 to 0.75644, about 2.3 rows), and it is also the only change that introduced a model with low prediction agreement — exactly as the diagnosis predicted.

6.5 Error analysis

Measure	Value	Interpretation
Mean confidence when correct	0.9350	decisive when right
Mean confidence when wrong	0.3643	confidence is meaningfully calibrated
Correct answer within top three	100.0%	MAP@3 loses nothing to top-3 misses
Correct answer at rank one	99.7%	residual loss is rank-two placements

Table 6.3 — Error analysis of the fine-tuned model on 300 validation rows.

- The confusion matrix shows errors spread across letters rather than concentrated on one option; despite B and C being more frequent in training the model does not over-predict them, so there is no frequency or positional bias.
- The confidence separation matters for three reasons: the probabilities carry information rather than being arbitrary; the model's uncertainty when its first choice is wrong is why the correct answer usually still appears at rank two; and well-calibrated probabilities make averaging across seeds and families sound.
- Reading individual failures, the predicted option addresses the same topic and reads plausibly — for one item on crystallographic point groups the model answered 27 against a true 32, with confidence 0.364. The residual errors are genuine discrimination failures, not preprocessing defects. The first-letter distribution of the final submission (A=91, B=115, C=116, D=92, E=86) is spread across all five options, confirming predictions are not degenerate.

7. Conclusion and Future Work

7.1 Key learnings

- Similarity is not correctness. Lexical similarity scored 0.1635 and semantic similarity 0.2567, at or below chance, because distractors are deliberately on-topic. Establishing this in the first hour redirected the entire project.
- An implausibly good result is a signal to investigate, not to celebrate. A perfect validation score from a 644,737-parameter model led directly to the central finding of this project.
- Shortcut learning is distinct from overfitting, and diagnosing it requires an architectural experiment rather than a metric — a model structurally incapable of reading the question provided evidence no score alone could supply.
- When validation is contaminated the leaderboard becomes the only trustworthy signal, and differences must be judged against the resolution of the metric — with 500 test rows anything below 0.004 is roughly two rows and cannot support a conclusion. Establishing this prevented hours of unproductive tuning.
- Ensembling requires diversity, not multiplicity. Seed bagging and cross-validation combine models agreeing above 99% and cannot help; measuring agreement before blending turned a guess into a prediction, and ELECTRA at 83% was the one change that broke the plateau.
- Parameter-efficient fine-tuning is remarkably effective: training 0.79% of parameters reached 0.9983 validation MAP@3, and the small optimizer footprint made full-precision training viable when mixed precision failed.

7.2 Challenges faced

Challenge	Resolution
Training loss exploded to 543	diagnosed as large gradients from the fresh classifier head; fixed with warmup and gradient clipping
bf16 rejected, fp16 failed under LoRA; torchao broke peft imports	trained in float32, affordable because LoRA trains 0.79% of parameters; torchao uninstalled in the first cell
Validation scores untrustworthy and ensemble weight search saturated at 1.0	prompt-blind experiment quantified the contamination; all decisions and the blend weight moved to the leaderboard
Score plateaued across six configurations	diagnosed via prediction agreement; resolved by introducing a different pretraining family

Table 7.1 — Significant challenges and how each was addressed.

7.3 Areas for improvement

- Prompt-dropout regularisation: include training examples with the question removed and a uniform or random target, directly penalising the ability to answer without reading the question. Unlike option shuffling this attacks the mechanism rather than the position, and the label-smoothing experiment showed agreement can be moved.
- Counterfactual augmentation: pair each question with distractors drawn from other questions, breaking the association between writing style and correctness in the training distribution.
- Systematic architecture diversity: since cross-family blending was the only effective lever, a principled search across pretraining families — selecting candidates by measured disagreement rather than individual accuracy — is the most promising direction. Rank-based rather than probability-based ensembling would also prevent a highly confident model from dominating the average.
- A genuine external knowledge corpus, which would let the retrieval pipeline implemented in the notebook contribute facts the model does not hold; the current knowledge base, built from training answers, contains nothing a test question can use.